A Practical Closed-Box Adversarial Attack on Graph-Learning-Based Models for Malware Detection

Kai Tan, Dongyang Zhan, *Member, IEEE*, Haining Yu, Hongli Zhang, *Member, IEEE*, Bei Zhao, and Xiaojun Jia

***Abstract*—Analyzing malware based on API call sequences is effective, as these sequences reflect the dynamic execution behavior of malware. The latest advancements in deep learning have facilitated the use of these techniques to mine useful information from API call sequences. However, existing methods primarily focus on extracting features from raw sequences, which may not capture critical information effectively, particularly in multi-process malware due to the interleaving issue of API calls. Recent research has introduced a graph-based method for embedding malware detection APIs. This approach offers two main advantages over existing embedding methods: graph models are invariant and can capture intra-process and inter-process behaviors more concisely and accurately. Given the inherent susceptibility of Graph Neural Networks (GNNs) to adversarial attacks, these models are vulnerable to such threats. To evaluate the security risks of existing malicious software detection systems based on API call graphs under closed box settings, a heuristic intent masking algorithm has been designed to alter intentions that capture fine-grained malware behavior. This involves an iterative selection of important nodes based on an importance measure between nodes, followed by masking these nodes using heuristic paths. The malware file is then reconstructed based on an optimized transformation sequence that maintains the original file's semantics, adhering to program format constraints. Our approach systematically evaluates four state-of-the-art graph-based malware detection systems under a closed-box setup. The evaluation results indicate that it achieves a high attack success rate. Additionally, the adversarial samples generated are able to maintain their original functionality.**

***Index Terms*—API Call Graph, adversarial attack, graph neural network, malware detection.**

I. Introduction

MALWARE poses a significant threat to the security and integrity of information systems worldwide [1], [2], [3]. As digital infrastructures become increasingly interconnected and reliant on networked data, the potential damage from malware infections escalates, impacting everything from personal data privacy to the operational stability of critical systems. The evolving complexity and sophistication of malware variants necessitate advanced detection methods to preemptively identify and mitigate these threats before they can inflict widespread harm.

Malware detection methods are primarily categorized into static and dynamic analysis. Static analysis [4], [5], [6] extracts specific features directly from executable files, such as headers, opcode sequences, and static API calls. However, techniques like obfuscation and metamorphism can diminish the efficacy of static analysis. In contrast, dynamic analysis [7], [8], [9], which extracts behavioral information such as network traffic, registry operations, and system calls, runs the programs in an isolated environment. This method's ability to observe execution behavior allows it to effectively counter various code obfuscation techniques.

The dynamic analysis executes malware in a controlled setting, extracting useful runtime behaviors such as communication packets, API calls, and system calls [10], [11]. As malware eventually engages in malicious activities, such as communicating with controllers, downloading additional malware, and accessing privileged files, these behaviors help distinguish malicious from benign software or differentiate between malware families. Consequently, machine learning (ML) and deep learning (DL) techniques are commonly applied to detect and classify malware based on API call sequences, which reflect the runtime behavior of programs.

Despite significant advancements in malware analysis, modern malware frequently employs multi-process mechanisms, which add layers of complexity to detection efforts [12]. These multi-process mechanisms are employed for various reasons, including enhancing operational efficiency, evading detection systems, and delegating distinct tasks to separate processes. Such strategies result in the corresponding API/system calls across multiple processes. Due to the intricacies of execution logic and CPU scheduling, API/system calls from different processes may become interleaved. This interleaving leads to a non-deterministic sequence of API/system call events, complicating the analysis process [9]. The randomness introduced by interleaving significantly challenges the effectiveness of

Received 22 December 2025; revised 30 March 2026; accepted 18 June 2026. Date of publication 23 June 2026; date of current version 1 September 2026. This work was supported in part by the National Natural Science Foundation of China under Grant 62302122 and Grant 62172123, in part by the National Key R&D Program of China under Grant 2025QY2824, and in part by the Heilongjiang Province Science and Technology Innovation Base Reward Project under Grant JD25A017. *(Corresponding author: Dongyang Zhan.)*

Kai Tan, Dongyang Zhan, Haining Yu, and Hongli Zhang are with the School of Cyberspace Science, Harbin Institute of Technology, Harbin 150001, China (e-mail: tankai@hit.edu.cn; zhandy@hit.edu.cn; yuhaining@hit.edu.cn; zhanghongli@hit.edu.cn).

Bei Zhao is with Mobile Design Institute, Beijing 100080, China (e-mail: zhaobei@cmdi.chinamobile.com).

Xiaojun Jia is with Nanyang Technological University, Singapore 639798 (e-mail: jiaxiaojunqaq@gmail.com).

Digital Object Identifier 10.1109/TDSC.2026.3706542

conventional malware detection and classification techniques, which predominantly rely on extracting features from linear sequences of API/system calls.

To overcome these challenges, recent studies [13], [14], [15], [16], [17] have turned to graph modeling techniques to decode the obscured behaviors inherent in complex malware operations. By employing graph-based representations, API/system calls are organized into a graph that reflects the interactions and behavioral patterns of processes. After representing these interactions as a graph, Graph Neural Networks (GNNs) are then used to perform graph classification tasks for malware detection. This approach not only captures structure information among the API/system call sequences but also resolves the interleaving issue, thereby enhancing the ability to detect and analyze both apparent and potential malicious activities embedded within the malware.

Although GNNs have demonstrated impressive performance in graph modeling tasks, a substantial body of recent researches indicate that GNNs are vulnerable to adversarial attacks [18], [19], [20], [21], [22], [23], [24], [25], [26], [27], [28], [29], where small, deliberately introduced perturbations can significantly degrade their performance. The primary objective of graph adversarial attacks is to subtly manipulate original graph data to diminish the effectiveness of GNNs. In light of these risks, this paper presents an adversarial attack in a realistic closed-box setting that effectively generates adversarial malware samples capable of evading graph-learning-based malware detection systems, thereby emphasizing the need for enhanced security measures.

In real-world scenarios, adversarial attacks against graph-based malware detection models represent a complex challenge far beyond merely targeting isolated models [30]. Instead of simply manipulating inputs to evade a single detection model, such attacks involve intricate strategies to manipulate call graphs and generate practical adversarial samples. There are two key challenges in creating such adversarial samples effectively:

1) Maintaining Functionality with Minimal Perturbation: General graph adversarial attacks face practical application challenges, particularly when manipulating malware's call graphs to create perturbation features. These challenges can be categorized into three main types: a) Functionality preservation and perturbation mapping. b) Global-level-graph information utilization. c) Imperceptibility of modifications. The first challenge involves directly modifying node features or graph structures [19] to produce adversarial features, which is impractical because it compromises the original program's functionality. In addition, perturbation mapping also proves challenging when attempting to generate corresponding adversarial graphs. The second challenge is the difficulty in utilizing global graph-level classification information for generating effective local node-level adversarial examples [21]. Therefore, adversarial perturbations are typically confined around single nodes or disproportionately affect high-degree nodes, reducing their effectiveness and detectability.

The third challenge is that adversarial modifications must remain covert to avoid detection. Minimal perturbation is particularly important in the context of dynamic malware detection. In realistic threat models, noticeable changes to the API/System call graph can easily be flagged by defenders or trigger manual inspection. Small modifications are more practical and stealthy in real-world attack scenarios. Analytically, using minimal modifications enhances interpretability, as it allows clearer attribution of misclassification to specific behavioral perturbations. Prior work on adversarial attacks against behavioral malware classifiers [31] similarly emphasizes the goal of achieving minimal yet effective graph perturbations.

2) Efficient Search in a Closed-Box Setting: Our review of existing state-of-the-art closed-box adversarial attacks includes gradient-based attacks using surrogate models [32], [33], evolutionary algorithms [34], [35], and reinforcement learning [36], [37]. However, attacks using these methods are computationally expensive due to the vast and discrete nature of malware spaces.

To our knowledge, this is the first framework that completes the full loop from graph-level adversarial perturbations to verifiable, semantics-preserving, executable-sample evasion against graph-learning malware detection under a closed-box setup. Our approach first derives call graphs from malware's dynamic execution and identifies the most influential subgraphs for classification. By focusing on these critical subgraphs, we reduce the search space for effective modifications. We then apply reinforcement learning techniques to execute rewiring operations on these subgraphs while preserving program functionality, and map the modified structure back to source code, enabling evasion of real-world malware detection systems.

In summary, our contributions in this paper are outlined as follows:

- *Graph-learning-based API/syscall malware detection adversarial attack:* We propose a practical adversarial attack approach for Graph-learning-based malware detection in closed-box settings, which preserves malware functionality while effectively evading detection.
- *Adversarial modification efficiency optimization:* We extract critical subgraphs and employ heuristic random walks to reduce the large discrete modification search space.
- *Malware reconstruction:* A transformation approach is introduced, which maps graph modifications back to the source code.
- *Extensive evaluation:* Through extensive experimentation, we demonstrate Attack's efficacy, revealing the inherent vulnerabilities of current detection methods.

The remaining sections of the paper are structured as follows. The background is introduced in Section II. Section III describes the overall framework and its implementation. The evaluation of our approach is performed in Section IV. Section V summarizes the related work. Section VI and Section VII discuss and conclude this paper.

II. Preliminaries & Threat Model

In this section, we introduce the target detection models, threat model of our approach, and the perturbation analysis.

A. Call Graph Construction

The malware detection process begins with the dynamic analysis of executables, focusing on collecting API/syscall sequences. Then it transforms these sequences into graphs to

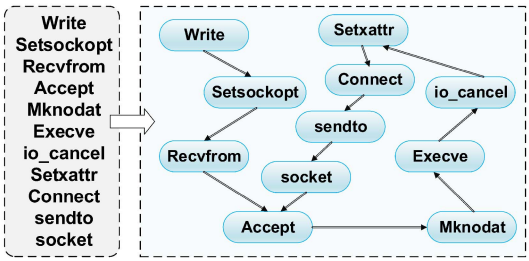

Fig. 1. Graph generation example.

capture the structural dependencies between API/system calls. Formally, given a set of API/system call sequences, each program is represented by a graph $G = (A, X, Adj)$, where A is a set of nodes and each $a \in A$ denotes a unique API/system call. The attribute matrix X is defined as $X = \{\ldots, x_i, \ldots\}$, where x_i specifies the attributes of the i-th node. The adjacency matrix for graph G is denoted as $Adj \in \mathbb{Z}^{N_G \times N_G}$, where N_G is the total number of API/system nodes in the graph. This API/system call graph structure effectively preserves all calls along with their sequence information, facilitating detailed behavioral analysis. It can also capture broader interactions, including those that span across processes, thereby providing a more comprehensive view of program behavior.

Fig. 1 shows a segment of the syscall sequence from a sample of malware. The sequence contains 11 system calls. After transformed into a directed graph, it illustrates how each system call not only follows sequentially but also connects logically to depict the flow and dependencies in malware's operations. For instance, initial write operations could set up the environment, followed by setsockopt to configure network settings, leading into recvfrom and accept to establish and accept network connections, which then enable the subsequent connect and sendto actions for active data transmission and command execution.

B. Preliminaries on Graph-Learning-Based Detection

The malware detection process begins with the dynamic analysis of executables, focusing on collecting API/syscall sequences, as shown in Fig. 2. Feature engineering is applied through the function $\phi : \mathcal{Z} \to \mathcal{X}$, which transforms a given executable z from the problem-space $\mathcal{Z}$ (i.e., $z \in \mathcal{Z}$) into a feature vector x in the dynamic graph-based feature-space $\mathcal{X}$ (i.e., $x \in \mathcal{X}$). This transformation involves capturing API/syscall call sequences and structuring these interactions into a graph model that accurately depicts the behavioral relationships and patterns within the system.

The dynamics of API/syscall interactions, encapsulated as graph-based features, are crucial for training machine learning (ML) or deep learning (DL) models for malware detection. The detection function $f : \mathcal{Z} \to \mathcal{Y}$ utilizes these graph representations to both train and predict. For a given executable z, f assigns a label y within the label-space $\mathcal{Y}$ where $\mathcal{Y} = \{0, 1\}$; here, $y = 0$ denotes benign software ("goodware") and $y = 1$ indicates malware.

This task mirrors a graph binary classification challenge, pairing graphs with labels $\{G_i, c_i\}_{i=1,\ldots,N}, c \in \{0, 1\}$. The goal is to learn a mapping function $f : G \to C$ that includes only one fully connected layer in its architecture. For a graph $G_i = (A_i, X_i)$ with n_i nodes, the graph classification process is described by:

$$h_i^{(0)} = X_i, \quad h_i^{(l)} = f_{\text{GNN}}(h_i^{(l-1)}; \Theta_l), \quad l = 1, 2, \ldots, L \tag{1}$$

$$h_i = \text{pooling}(h_i^{(L)}), \quad z_i = W h_i + b \tag{2}$$

where $h_i^{(l)} \in \mathbb{R}^{n_i \times D_l}$ represents the hidden node embeddings at the l-th GNN layer, i.e., graph convolution [38]. $W \in \mathbb{R}^{C \times D_L}$ and $b \in \mathbb{R}^C$ are parameters of the output layer, with L indicating the number of graph convolutions.

The objective function for graph classification is formulated as follows:

$$l(f_\Theta(z), c) = -\left(c \log(p(z)) + (1 - c) \log(1 - p(z))\right) \tag{3}$$

where $l(\cdot, \cdot)$ is a loss function, specifically cross-entropy. This loss function effectively calculates the probability that an executable is malware ($p(z)$) and its complement, the probability of being benign ($1 - p(z)$), based on the binary classification nature of the model. The cross-entropy loss optimizes these probabilities to improve prediction accuracy, directly enhancing the detection system's capability to interpret complex behavioral data, thereby improving the accuracy and reliability of malware identification.

After the model is trained, it is deployed to detect new executables. Using the learned patterns and relationships, the model evaluates new executables by applying the trained function f, predicting their labels as either benign or malware.

C. Threat Model

We explain the threat model based on the attacker's goals and capabilities.

Adversary's Goal: The primary goal of the adversary is to generate an adversarial malware file z_{adv} derived from an original malware $z \in \mathcal{Z}$ (i.e., $f(z) = 1$), with minimal efforts. The adversary aims to modify the API/system call graph generated from z, along with corresponding code changes, to create z_{adv}. The newly generated $z_{\text{adv}} \in \mathcal{Z}$ is designed to not only evade the target learning-based malware detection system f (i.e., $f(z_{\text{adv}}) = 0$), but also to preserve the same semantics and functionality as z. This requires precise manipulations that maintain the integrity of the malware's original intent while deceiving the detection mechanisms.

Adversary's Capability: The adversary performs a closed-box attack on the target detection system. This indicates that the attacker does not possess prior information about the target system's training data, feature set extraction, the learning algorithm with its parameters, or the weighted model architecture. The capability of the adversary is limited to understanding that the target system is based on dynamic analysis, specifically focusing on API call graphs.

After inputting z, the adversary knows only the prediction label $f(z)$ and its probability $g(z)$. The attacker trains a surrogate

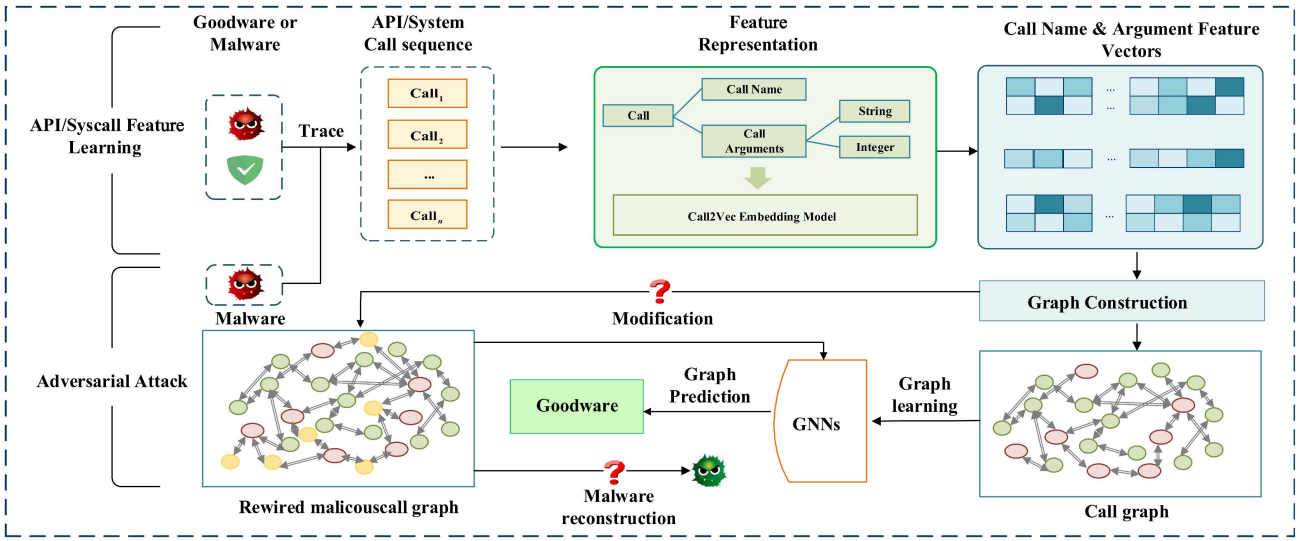

Fig. 2. Target Model & Problem Statement. The attack aims to evade a GNN-based malware detector by making adversarial modifications to the call graph that can be reflected in the malware, producing practical adversarial samples.

model by querying the target model and collecting input-output pairs. Similar to previous works [39], [40], this strategy enables the approximation of the target model's behavior without requiring access to its internal components. Additionally, to ensure the creation of realistic adversarial malware files, the adversary is capable to modify the malware source code and generate executable files based on the modified API/system call graph. Note that this is an evasion attack operating strictly at test time, not a poisoning attack—the adversary does not interfere with the model's training process.

D. Problem Statement

The primary problem is to alter a call graph and corresponding source code to generate a realistic adversarial malware file $z_{\text{adv}} \in \mathcal{Z}$, designed to be misclassified as goodware while retaining the original functionality of the malware z. This aims to test the robustness of current malware detection systems by creating adversarial files that mimic malicious behavior yet are classified as benign. The principal formulation is as follows:

$$\text{argmin}_T \; f(z_{\text{adv}}) \tag{4}$$

where the objective is to minimize the detection function $f(z_{\text{adv}})$ ensuring that z_{adv} is misclassified as goodware.

The minimization targets the misclassification of z_{adv} as goodware, under the constraints:

$$f(z) = 1, \quad f(z_{\text{adv}}) = 0, \quad z_{\text{adv}} = T(z) \in \mathcal{Z} \tag{5}$$

Here, the original file z is identified as malware ($f(z) = 1$), the adversarial file z_{adv} as goodware ($f(z_{\text{adv}}) = 0$), and z_{adv} results from a direct transformation T of z, aiming to evade detection while preserving functionality.

The transformations Γ are selected to ensure that the modifications to the API/system call graph retain the executable's functionality. The sequence of transformations is modeled as:

$$\Gamma = \Gamma_1 \circ \Gamma_2 \circ \cdots \circ \Gamma_n \in \mathcal{S} \tag{6}$$

As depicted in Fig. 2, in real attack scenarios, two challenges arise: maintaining program functionality and accurately reconstructing the malware to reflect perturbations effectively. These challenges are addressed with the following methodologies:

Graph Rewiring Attack: The Graph Rewiring Attack (GRA) modifies G by selectively deleting and adding edges, alongside introducing new nodes to preserve the original functionality of the program. This rewiring action involves removing a direct edge between nodes a_1 and a_2, and introducing a new intermediate node a_{int}. New edges are then added to connect a_1 to a_{int} and a_{int} to a_2.

Specifically, this process transforms the original graph $G = (A, X, D)$ into a new graph $G' = (A', X', D')$. The new edge set D' is given by $\mathbf{D}' = \mathbf{D} - \mathbf{E}_{\text{del}} + \mathbf{E}_{\text{add}}$, where $\mathbf{E}_{\text{del}}$ indicates the deletion of the edge (a_1, a_2) and $\mathbf{E}_{\text{add}}$ represents the addition of edges (a_1, a_{int}) and (a_{int}, a_2). The node set A' includes the new intermediate node a_{int}, and the feature matrix X' updates to incorporate features for a_{int} while retaining existing node features.

The objective of GRA is to subtly modify the graph's operational structure to minimize detectability by defenders. The formal optimization goal is:

$$\min_{\mathbf{G}'} \|\mathbf{G}' - \mathbf{G}\| \tag{7}$$

subject to constraints on the number of edges deleted (d_1) and added (d_2), and ensuring that the modifications do not compromise the program's functionality:

$$\|\mathbf{E}_{\text{del}}\| \leq d, \quad \|\mathbf{E}_{\text{add}}\| \leq a \tag{8}$$

Source Code Mapping: For Graph Rewiring Attacks, this structural modification is reflected in the system call sequence

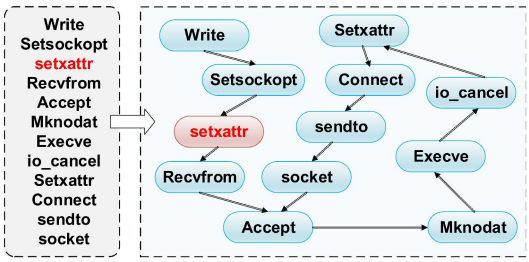

Fig. 3. Rewiring operation example.

by inserting a new system call a_{mid} between the system calls s_1 associated with a_1 and s_2 associated with a_2. Therefore, this modification cannot damage the functionality of the malware.

As shown in Fig. 3, in the system call sequence, if we plan to insert an `'setxattr()'` call between `'setsockopt()'` and `'recvfrom()'`, we remove the direct edge from `'setsockopt()'` to `'recvfrom()'` in the call graph and create new edges from `'setsockopt()'` to `'setxattr()'` and from `'setxattr()'` to `'recvfrom()'`. This operation effectively inserts `'setxattr()'` into the original call sequence.

Through this method, we achieve a mapping from modifications in the call graph to changes in the sequence. This strategy not only preserves the fundamental functionality of the program but also subtly alters the program's execution flow through well-designed rewiring operations. To translate these sequence modifications back into the program's source code, we utilize methodologies from previous study [10] that have demonstrated effective ways to reflect sequence changes in the source code, ensuring that the adversarial modifications are executable and maintain the malware's intended behavior.

III. System Design

A. Framework Overview

As illustrated in Fig. 4, our framework comprises several key phases. Our approach starts by assessing node significance within call graphs and extracting influential subgraphs. It then utilizes heuristic random walks to identify representative malicious paths. Reinforcement learning (RL) is applied to perform rewiring operations, facilitating adversarial attacks on the call graph while preserving the inherent functionality of the application. Finally, we map these graph-level modifications back to sequence-level changes and reconstruct the adversarial malware. Detailed explanations of each phase are provided in Section III-B and Section III-C for subgraph extraction and path identification, Section III-D for RL-based rewiring operations, and Section III-E for malware reconstruction.

B. Vulnerability Search of Subgraphs

The extraction of subgraphs is essential for pinpointing potential targets for perturbations within a graph. In the context of closed-box attacks, we train a surrogate model to evaluate the contributions of nodes to graph classification. This method is inspired by visual interpretability techniques such as Graph CAM [41], [42] and its variants like Score CAM [43], which effectively highlight nodes of high importance.

Graph CAM: Serving as a key technique for explainability in graph classification, Graph CAM utilizes the weight matrix of the output fully connected layer to represent the significance of each dimension's features for graph classification. This method constructs a heatmap matrix by projecting the weight matrix back to the node representation in the final graph convolution layer, thereby indicating each node's importance for graph classification. The computation of this heatmap matrix is formalized as:

$$LCAM = \text{ReLU}\left(h(L)W^T\right) \tag{9}$$

where $W \in \mathbb{R}^{C\times D_L}$ is the weight matrix from the output layer, and $h(L) \in \mathbb{R}^{n\times D_L}$ denotes the node representation in the final graph convolution layer. The element $W_{c,k}$ in the weight matrix signifies the importance of the k-th node for predicting the class label c.

To advance model interpretability within Graph Convolutional Networks (GCNs), we have developed a modification known as Graph Score CAM. While Graph CAM explains classification decisions by projecting final-layer node representations through linear weights, it relies on two assumptions: (1) node importance can be inferred linearly from final-layer activations, and (2) the last GNN layer is sufficient for explanation. However, modern GNNs often learn hierarchical and nonlinear representations across multiple layers, making Graph CAM less faithful to the true causal influence of nodes. To overcome these limitations, Graph Score CAM adopts a perturbation-based strategy inspired by Score-CAM in CNNs, estimating node importance through changes in prediction confidence. This provides class-specific, gradient-free, and layer-aware attribution that better reflects the actual decision process of GNNs.

Building on this intuition, we formally define the confidence score for node activations as follows.

Specifically, the confidence score α_l for node activations is determined by the change in the network's output before and after activation, expressed as:

$$\alpha_l = C(H_l) = f(X \circ P_l) - f(X_b) \tag{10}$$

where H_l denotes the node activations at layer l, and P_l represents the node activations post upsampling and normalization, calculated as:

$$P_l = s(\text{Up}(H_l)) \tag{11}$$

Here, $\text{Up}(\cdot)$ denotes the upsampling operation that scales H_l to match the input graph X, and $s(\cdot)$ is a normalization function mapping the activations to [0,1]. The operation $X \circ P_l$ represents their element-wise product, enabling a direct assessment of each node's contribution to the prediction.

Through the integration of confidence scores from each layer, the final heatmap matrix, termed L_Score-CAM, is computed:

$$\text{L\_Score-CAM} = \frac{1}{L}\sum_l \text{ReLU}(h^{(l)}\alpha_l) \tag{12}$$

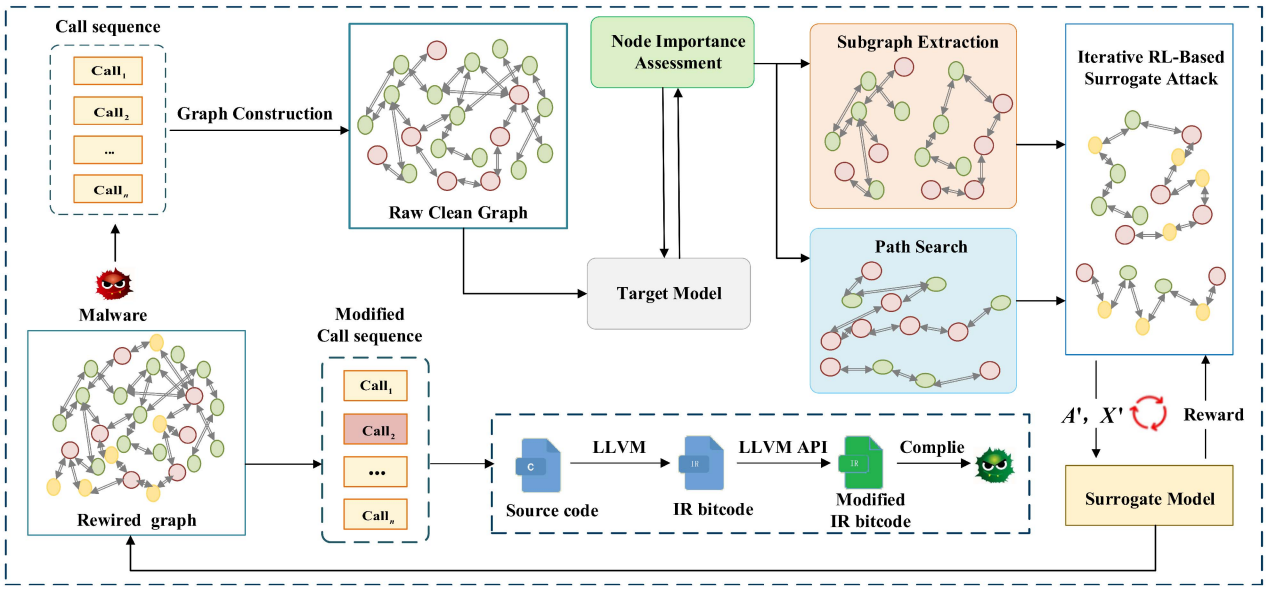

Fig. 4. The framework of our approach.

Algorithm 1: Efficient Extraction of Overlapping Connected Subgraphs from Top Ranked Nodes.

Input: Graph $G = (A, X, Adj)$ with n nodes; Graph CAM ; Proportion $p\%$.
Output: Set of extracted overlapping subgraphs $\{G_{\text{sub}_i} = (A_{\text{sub}_i}, X_{\text{sub}_i}, Adj_{\text{sub}_i})\}$.

1 **Select Top** $p\%$ **Nodes:** Sort nodes by CAM and select top $p\%$ nodes as A_{top};
2 **Extract Subgraphs: for each** *node* v *in* A_{top} **do**
  // Identify all nodes connected to v within A_{top}
3 Initialize C_v with v;
4 **for each** *node* u *in* $A_{top} \setminus \{v\}$ **do**
5 **if** *path exists between* v *and* u *in* Adj **then**
6 Add u to C_v;
7 **end**
8 **end**
  // Extract the adjacency and feature matrices for nodes in C_v
9 $Adj_{\text{sub}} \leftarrow$ Extract adjacency matrix for C_v;
10 $X_{\text{sub}} \leftarrow$ Extract feature matrix for C_v from X;
  // Store the constructed subgraph
11 Store the subgraph in $\{G_{\text{sub}_i}\}$;
12 **end**
13 **return** $\{G_{\text{sub}_i}\}$

where $z \in \mathbb{R}^C$ are the model's predicted log odds, and $h_v^{(l)} \in \mathbb{R}^{D_l}$ is the hidden embedding for node v in the l-th graph convolution layer. The calculation of Graph Score CAM not only provides a deep understanding of the graph structural data but also helps explain the decision-making process of graph neural networks, particularly in how node influence analyses across layers affect the final graph classification outcome.

The CAM heatmap matrix effectively highlights the significance of each node in graph classification tasks, aiding in the identification of key nodes crucial for the classification process. Subsequently, we extract critical subgraphs centered on these nodes as shown in Algorithm 1. For each graph G, a subgraph G_{sub} is formed by retaining $p\%$ of the top-ranked nodes A_{sub}, where $|A_{\text{sub}}| \leq p\%|A|$. These nodes are selected based on their ranking in the vector U_c of the CAM matrix, which corresponds to the predicted label c.

C. Vulnerability Search of Malicious Paths

Our algorithm leverages random walks [44] to extract critical API/system call paths from the graph, effectively masking the behavioral intentions of malware. By prioritizing nodes of high importance as starting points for walks, the algorithm not only efficiently controls the generation of paths but also prevents path explosion issues. It ensures that each path closely relates to malicious activities. This method accurately identifies and exposes the key operations of malware, thereby enhancing the implementation of adversarial attacks.

This algorithm efficiently explores a graph G by leveraging node importance metrics to guide random walks, thereby highlighting significant structural features within the graph. The algorithm initializes an empty path corpus $\mathcal{P}$ (Line 1) to store all paths generated across multiple random walks. Each walk begins by selecting a start vertex v_s based on the highest importance scores from the importance vector I (Line 3). This strategic choice ensures that each walk potentially uncovers influential pathways by starting from nodes of high significance.

For each walk, the algorithm iterates up to a predefined number of steps O (Line 6). At each step, it dynamically evaluates the current vertex's neighborhood (Line 7), filters potential next vertices based on their attributes and importance (Line 8), and computes transition probabilities (Lines 10-13). These

Algorithm 2: Heuristic Random Walk on Graph G.

Input: Graph $G = (V, E)$, Importance Vector I, Number of Walks n, Length of each Walk L
Output: Path Corpus $\mathcal{P}$

1 **Initialize:** $\mathcal{P} \leftarrow []$, $Prob \leftarrow []$;
2 **for** $k \leftarrow 1$ ***to*** n **do**
3 Select initial vertex v_s based on I (e.g., highest importance);
4 $path \leftarrow [v_s]$;
5 $current\_vertex \leftarrow v_s$;
6 **for** $step \leftarrow 1$ ***to*** L **do**
7 $neighbors \leftarrow$ Get Neighbors of $current\_vertex$;
8 $candidates \leftarrow$ Filter by dynamic criteria from $neighbors$;
9 **for each** v *in* $candidates$ **do**
10 $prob_v \leftarrow \frac{I(v)}{\sum_{u \in candidates} I(u)}$; `// Compute probability`
11 $Prob$.append((v, p_v));
12 **end**
13 $v_d \leftarrow$ Select next vertex based on $Prob$;
14 **if** v_d *is None* **then**
15 break ; `// End walk if no valid next vertex`
16 **end**
17 Append v_d to $path$;
18 $current\_vertex \leftarrow v_d$;
19 **end**
20 Append $path$ to $\mathcal{P}$;
21 **end**
22 **return** $\mathcal{P}$

probabilities dictate the walk's direction, favoring vertices that maintain or increase the path's overall importance and relevance. The probability p_v for selecting each vertex v is computed using the formula:

$$p_v = \frac{I(v)}{\sum_{u \in \text{candidates}} I(u)} \tag{13}$$

where $I(v)$ is the importance score of vertex v, and candidates are the neighboring vertices being considered for the next step.

The next vertex v_d is selected probabilistically (Line 14), ensuring that the walk is stochastic yet biased towards more important nodes. If no valid vertex is available (Line 15-17), the walk terminates early. Otherwise, the selected vertex is added to the current path (Line 19), and the walk proceeds.

After completing each walk, the resultant path is added to the corpus $\mathcal{P}$ (Line 21), and the process repeats for n walks. The algorithm returns the full path corpus $\mathcal{P}$, providing a comprehensive set of paths that collectively highlight critical and influential components of the graph.

D. Graph Adversarial Attack With Reinforcement Learning

1) Benign API/System Call Selection: To ensure that the inserted benign API/syscall node is both semantically neutral and contextually appropriate, we design a selection criterion based on two factors: (1) neutrality, achieved by excluding APIs associated with sensitive operations such as file modification, privilege escalation, or external network communication; and (2) context compatibility, ensured by selecting calls that perform harmless and lightweight actions (e.g., retrieving process information, querying system time, or accessing in-memory buffers) that naturally fit into the surrounding control-flow context. Formally, each inserted call must not modify any shared program state, produce any externally observable side effect, or alter the original malware's intended behavior. This strategy minimizes the risk of detection or removal by filtering mechanisms while maintaining a realistic execution profile.

2) Reinforcement Learning: Consider a graph G where an attacker T operates through a decision-making model $M = (S, A, P, R)$. Here, $A = \{a_t\}$ represents the ensemble of all permissible rewiring actions, and $S = \{s_t\}$ encompasses all attainable graph states post-rewiring. The transition dynamics, denoted as P, articulate how an action a_t alters the graph structure via the probability $p(s_{t+1} \mid s_t, \ldots, s_1, a_t)$. The function R stands for the reward mechanism, which allocates rewards based on the actions executed at each state.

The operational sequence of graph attack is characterized by the trajectory $(s_1, a_1, r_1, \ldots, s_M, a_M, r_M)$, initiating with $s_1 = G$. A significant aspect for the attacker involves mastering the decision-making for selecting an optimal rewiring action at any state s_t, achievable through a policy network that computes the likelihood $p(a_t \mid s_t, \ldots, s_1)$ and subsequently selects a rewiring action accordingly. This approach suggests that each decision at state s_t is contingent upon its preceding states, potentially complicating decisions due to long-term dependencies.

The decision-making process at any state s_t can effectively be isolated to consider only the present state, simplifying to $p(a_t \mid s_t, \ldots, s_1) = p(a_t \mid s_t)$. Consequently, the entire RL-based attack mechanism is formulated as a Markov Decision Process (MDP) [45], with the adoption of reinforcement learning techniques to refine decision-making strategies.

The essential components defining the reinforcement learning environment include:

- *State Space:* This includes all possible intermediate graph states that can be generated via various rewiring actions.
- *Action Space:* This consists of all valid rewiring operations. Since the attack strategy leverages benign APIs/syscall for inserting nodes and the critical subgraphs along with malicious paths have been predetermined, the scope for modifications is distinctly defined and constrained.
- *State Transition Dynamics:* When a specific action (rewiring operation) $a_t = \{v_{fir}, v_{thi}\}$ is applied at state s_t, the subsequent state s_{t+1} results from the deletion of the edge between v_{fir} and v_{thi} and the addition of two new edges linking v_{two} with both v_{fir} and v_{thi}.
- *Reward Design:* The principal objective for the attacker is to alter the classifier $f(\cdot)$'s prediction to differ from the initial classification. To encourage efficient and minimalistic graph modifications, a positive reward is assigned for successful attacks, while a negative reward is imposed for

each action undertaken. The reward mechanism is formalized as:

$$R(s_t, a_t) = \begin{cases} 1 & \text{if } f(s_t) \neq f(s_1) \\ nr & \text{if } f(s_t) = f(s_1) \end{cases} \tag{14}$$

where nr is the negative reward, designed to penalize each action step. This is similar to setting a flexible rewiring budget K, where $nr = -\frac{1}{K} = -\frac{1}{p \cdot |E|}$ is dependent on the graph's size.

- *Termination:* The attack concludes either when the number of actions reaches the budget K, or when the attacker successfully changes the graph's label.

3) Policy Network: In this section, we explore the design of a policy network that uses a Graph Convolutional Network (GCN) to identify the most effective rewiring actions within graph G. The focus is on selecting edges and benign APIs/syscalls essential for disguising adversarial modifications against malware detection systems.

The action decision process in our framework includes two key operations: firstly, an edge $e_t = (v_{e1}, v_{e2})$ is selected from the current graph state s_t's edge set, representing a potential rewiring connection. Secondly, after choosing an edge, a contextually appropriate benign API or syscall v_{benign} is selected from a predefined set of benign nodes N_{benign}, ensuring minimal detectability by malware detectors.

The action policy $p(a_t \mid s_t)$ combines the probabilities of these actions:

$$p(a_t \mid s_t) = p_{edge}(e_t \mid s_t) \cdot p_{insert}(v_{benign} \mid e_t, s_t) \tag{15}$$

The edge selection probability is determined by processing the combined features of its constituent nodes through a GCN, formalized as:

$$p_{edge}(\cdot \mid s_t) = \text{softmax}(\text{MLP}(E_{s_t} \mid \theta_{edge})) \tag{16}$$

where E_{s_t} is the feature representation of edges in state s_t, transformed by an MLP.

Following the selection of an edge, the benign insertion network chooses a suitable benign API/syscall for insertion at the selected edge to rewire the graph. This choice aims to minimize detection risk while maintaining operational functionality:

$$p_{insert}(\cdot \mid e_t, s_t) = \text{softmax}(\text{MLP}(B_{s_t} \mid \theta_{insert})) \tag{17}$$

where B_{s_t} represents the contextual features of benign APIs/syscalls compatible with the edge environment.

This strategy ensures that each selected action—both edge selection and benign insertion—effectively supports the adversarial objective, reducing the malware's detectability while preserving its functionality through subtle graph modifications.

4) Proposed Framework: Our framework employs a Graph Convolutional Network (GCN) to derive node and edge embeddings from the current graph state s_t. These embeddings inform the policy networks, enabling them to identify the most advantageous rewiring actions. During the rewiring process, we explicitly enforce the constraints $\|\mathbf{E}_{\text{del}}\| \leq d$ and $\|\mathbf{E}_{\text{add}}\| \leq a$ by terminating the modification procedure once either limit is reached, thereby ensuring that the modified graph $\mathbf{G}'$ remains structurally similar to the original $\mathbf{G}$. Upon selecting an action, the policy network orchestrates the rewiring of s_t, transitioning the system into a new state s_{t+1}. This subsequent state is evaluated by querying a closed-box classifier to obtain a prediction $f(s_{t+1})$, which is then compared against the initial prediction $f(s_1)$. This comparison forms the basis for computing the reward. To optimize the policy network, we apply a policy gradient method [46], aiming to maximize these rewards while strictly adhering to the predefined limits on rewiring operations.

E. Malware Reconstruction

As described in section II-D, rewiring a call graph can be seen as inserting new syscalls into the original sequence. Subsequently, we modify the source code of the malware to realize these sequence modifications. We adopt the method used in [10]. We utilize LLVM [47] to modify the source code, due to its capability to convert code from multiple languages into a streamlined Intermediate Representation (IR) bitcode. This format consists largely of basic blocks and directives, simplifying the analysis and modification tasks.

Initially, we compile the source with debugging enabled to produce an executable. We then deploy the strace tool to log a series of system calls while capturing the stack functions' file offsets. These offsets are subsequently correlated to specific source locations through the addr2line tool, yielding precise code locations for each syscall traced.

Following this, we compile the source into LLVM IR bitcode, incorporating debugging data to preserve line numbers and file identifiers within the IR statements. We analyze these statements to pinpoint any associated with system calls. Upon identifying the relevant statements, we engage the LLVM API to seamlessly integrate a specially crafted function ("insertSyscall") before each statement tied to a system call, based on the previously established code locations.

The final step involves recompiling the adjusted IR bitcode into an executable, thus producing a modified system call sequence that accurately reflects the implemented changes.

IV. Evaluation

To validate our approach, we evaluate the performance of our approach by answering the following research questions:

- *RQ1. Effectiveness Analysis:* How effective is our approach against the state-of-the-art graph-learning-based detection models?
- *RQ2. Impacting Factors:* How is the modification efficiency of our approach?
- *RQ3. Modification Efficiency Comparisons:* How is the modification efficiency of our approach?
- *RQ4. Generalizability:* Is our approach effective against a variety of malware detection systems that utilize API/system call sequences?
- *RQ5. Practicality Assessment of Attacks:* What is the feasibility of our approach?
- *RQ6. Effectiveness under Reduction Strategies:* How effective is our approach when facing feature reduction strategies?

A. Environment

The hardware configuration of our system includes an AMD Ryzen 7 5800 8-Core Processor 3.40 GHz, NVIDIA 3060 graphics card, and 32.0 GB of RAM. we have released the key components of our implementation, including the core modules of the proposed method, on GitHub: `https://github.com/embrace000/Graph-attack`

B. Dataset and Sequence Generation

Our evaluation methodology encompasses two public datasets and one synthesized dataset:

1) The initial dataset, the ADFA-LD [48], comprises a comprehensive set of system call traces aimed at assessing intrusion detection systems (IDS) within Linux environments. It includes both normal and malicious behaviors, simulating a variety of attack scenarios. This dataset serves as a critical resource for researchers to develop and test IDS models, thus enhancing cybersecurity initiatives and deepening the understanding of IDS techniques in Linux-based systems.

2) To further test the efficacy of our approach, we modify malware on the Linux x86 platform. As the corresponding source code of the ADFA-LD dataset is not provided, we query Exploit-DB using the keywords "Linux" and "local" and filter for PoCs written in C/C++. From these, we select only those that can be compiled successfully, obtaining 106 usable PoCs. These PoCs are compiled into executable files and their system call sequences are traced using the strace tool. The traced sequences, obtained via a fixed-window grouping approach with 2500 system calls, are labeled as attack samples and incorporated into the ADFA-LD dataset.

3) The final dataset used is the AndroCT [49], specifically designed for detecting android malware. It encompasses over 35,974 Android applications, collected from 2010 to 2019, representing a wide spectrum of both benign and malicious applications. The dataset is characterized by API call sequences.

For data processing, we employed a fixed window grouping method to generate behavior sequences from these datasets. In the labeling phase, sequences indicative of malicious behavior were labeled as "positive", while benign sequences received a "negative" label.

C. Evaluation Method

We introduce several commonly used metrics in the context of adversarial attacks:

Success rate (SR): The proportion of successful adversarial attacks, where the prediction of a classifier is changed from the malicious class to the benign class. Mathematically, SR is defined as:

$$SR = \frac{\text{Number of successful attacks}}{\text{Total number of attacks}} \tag{18}$$

The Perturbation Rate (PR) in the context of graph-based adversarial attacks measures the extent of modifications made to the original graph structure, particularly through node additions and edge rewirings. This metric quantifies the degree of distortion introduced to the graph by computing the average ratio of the number of perturbed elements (new nodes and rewired connections) to the total number of original connections. The modified formula for PR can be expressed as follows:

$$PR = \frac{1}{N}\sum_{i=1}^{N}\frac{\text{Number of perturbed elements in graph } i}{\text{Total number of original edges in graph } i} \tag{19}$$

where *perturbed elements* may include both newly added nodes and rewired connections, and *original edges* refer to the count of connections present before modifications.

D. Target Model

We choose four graph-learning-based models:

1) GCN-based model [50]: John et al.(2020) proposes an Android malware detection method using Graph Convolutional Networks (GCN). By modeling system calls sequences as graphs, this method leverages the structural dependencies between system calls to identify malicious behaviors.

2) GIN-based model [51]: Deng et al. (2023) propose "Enimanal", a framework for IoT malware analysis across different architectures using graph isomorphism networks (GIN). This approach enhances semantic understanding of system calls by incorporating details from the Linux Programmer Manual. Through constructing an attributed system call graph, Enimanal significantly improves malware classification effectiveness.

3) GAT-based model [13]: Feng et al. (2024) present DawnGNN, a novel framework for Windows malware detection that leverages graph neural networks. It uniquely incorporates official API documentation into the model, enhancing the semantic representation of API calls through a pre-trained BERT model. The encoded API calls are then processed using a Graph Attention Network (GAT) classifier, which efficiently captures and utilizes the relationships between API calls to improve detection accuracy.

4) API2Vec++ [12]: API2Vec++ is a graph-based API embedding method that significantly enhances malware detection and classification. This method employs a heuristic random walk algorithm across Temporal Process Graphs (TPG) and Temporal API Property Graphs (TAPG) to generate representative paths. These paths are then embedded using a BERT model, focusing on capturing the nuanced behaviors of multi-process malware.

Among the compared models, GIN, GCN, and API2vec++ do not provide public access to their pretrained models or datasets. As such, we reimplemented these models based on the technical details in their respective papers, ensuring faithful reproduction. For API2vec++, we used the official open-source codebase to preserve original configurations.

E. Baselines

We compare our approach with representative adversarial attack baselines which perturb graphs from various perspectives

1) Random [52]: Randomly selects nodes to perturb and edges to insert/delete.It requires minimal information for attack. Although it is simple, sometimes it can achieve a good attack rate.

TABLE I
THE SUCCESS RATE FOR DIFFERENT CLOSED-BOX SCENARIOS

Dataset	Attack	GCN		GIN		GAT		API2Vec++	
		w/ prob	wo/ prob	w/ prob	wo/ prob	w/ prob	wo/ prob	w/ prob	wo/ prob
AndroCT	Random	23.42	20.30	20.15	19.24	17.55	15.88	13.48	13.33
	ReWatt	25.61	24.77	22.76	21.41	20.95	19.87	17.65	16.96
	PageRank	30.34	28.56	26.54	25.62	24.36	23.85	21.41	21.05
	RL	32.10	30.95	28.94	27.86	26.85	25.94	23.88	23.41
	CAMA	32.92	31.84	30.15	29.42	28.08	27.36	24.92	24.47
	Ours	35.45	35.32	33.27	33.25	31.41	30.52	28.45	27.67
ADFA-LD	Random	32.44	31.35	28.57	26.41	24.33	23.54	21.91	20.85
	ReWatt	35.45	33.32	30.15	28.74	27.65	27.54	26.71	24.92
	PageRank	41.35	39.85	37.21	36.98	35.41	34.63	33.41	32.95
	RL	42.02	40.78	38.15	37.24	35.52	34.71	33.18	33.46
	CAMA	42.28	40.96	39.85	38.12	36.44	35.63	34.94	34.28
	Ours	45.61	44.32	42.48	41.74	39.91	38.15	37.27	36.97

2) ReWatt [53]: ReWatt conducts rewiring operations to perform structure attacks and uses reinforcement learning to find the optimal rewiring operations. ReWatt's rewiring operation involves removing an edge between a node and its direct neighbor, and then connecting that node to a more distant neighbor instead. While this method effectively alters the graph's structure, it can also disrupt the original program functionality by changing the direct connectivity between nodes.

3) PageRank [24]: PageRank is a graph centrality metric. Here, we attack nodes with the top-ranked Pagerank scores.

4) RL [19]: It propose a reinforcement learning based closed-box attack on graph-structured data that focuses on modifying graph edges. Their method adopts a hierarchical Q-function to decompose the large action space of edge addition and deletion, enabling efficient training using only the prediction feedback from the target classifier.

5) CAMA [54]: CAMA identifies high-importance nodes for graph-level prediction and restricts structural perturbations to these nodes. Specifically, it removes edges between highly similar node pairs and adds edges between low-similarity yet high-importance node pairs, where similarity is computed using cosine similarity of intermediate-layer node embeddings.

F. Results

1) RQ1: Effectiveness Analysis.: In our extensive evaluation detailed in this research, we meticulously analyzed the performance of our approach against a robust set of baseline adversarial attacks. Utilizing the above two datasets provided a broad and challenging testing ground for our algorithms. The results, systematically illustrated in Table I, demonstrate that our approach can achieve a high success rate for all detection models, including GCN, GIN, GAT, and API2Vec++.

Our approach rigorously tested in two scenarios, both with and without probability adjustments—to showcase its ability under different conditions. In scenarios probability adjustments, the surrogate model can mimic the target model's output classification probabilities. Our approach achieves high attack success rates, exceeding 37.27% across all models compared to other methods. It illustrates our approach's ability to utilize classification probabilities and achieve high efficacy.

Moreover, in the scenarios without probability adjustments, the surrogate model can only mimic the target model's output classification labels, our approach maintained robust performance. The success rates surpass 36.97% across various models, highlighting the method's capability to effectively operate even under constrained informational feedback. This aspect is critical in environments where attackers have limited ability to gauge the direct impact of their actions on the system.

Compared to other methods, our approach demonstrated its superiority by manipulating crucial subgraphs for evading detection. The approach alters node connectivity through rewiring operation that significantly obfuscates malicious patterns without disrupting the operational functionality of the malware, thus ensuring successful execution. This manipulation of graph features allows our approach to consistently bypass state-of-the-art malware detection systems, which rely on graph-based features for malware classification.

Furthermore, the adaptability of our approach was tested across different graph detection models, demonstrating the transferability of our adversarial attacks on graph-learning based detection models.

Overall, the experimental results validate the ability of our approach to conduct effective and sophisticated adversarial attacks in closed-box settings.

2) RQ2: Impacting Factors.: We conduct a comprehensive ablation study to dissect the critical components involved in our graph manipulation strategy. This study is designed to elucidate the individual and combined contributions of the CAM graph, node paths, and the utilization of only benign nodes for rewiring operations.

Impact of key parts in our approach: We analyzed the impact of key components in our approach on performance.

1) Removal of Graph Score CAM (ro-GC): In this ablation scenario, we eliminate the Graph Score CAM from our process, which provides importance estimation of crucial activation nodes that are significant for adversarial manipulation. This module is instrumental in identifying strategic points for

TABLE II
ABLATION RESULTS ON THE SUCCESS RATE OF OUR APPROACH

Dataset	Target	ro-GC	ro-MP
AndroCT	GCN	33.85	32.71
	GIN	31.17	32.51
	GAT	30.48	29.86
	API2Vec++	26.74	27.21
ADFA-LD	GCN	43.14	42.84
	GIN	40.55	39.74
	GAT	36.56	35.97
	API2Vec++	34.88	34.74

effective rewiring. By removing it, we intend to evaluate how the absence of these guided insights affects the overall effectiveness and subtlety of the attacks. We also compare with replacing it by Graph CAM to isolate the benefit of our perturbation-based scoring.

2) Removal of malicious Paths (ro-MP): This part of the study focuses on omitting the malicious paths, which are usually derived through heuristic approaches to establish potentially vulnerable or impactful routes in the graph. These paths are critical for planning and executing structured rewiring strategies. By excluding malicious paths, our goal is to determine this impact on the effectiveness of graph perturbations, assessing whether modifying critical malicious paths are significantly effective.

As shown in Table II, our ablation study reveals that both the CAM graph and malicious paths play pivotal roles in the efficacy of our adversarial attacks on graph-based systems. When the Graph Score CAM was removed, the performance declined by 1.9%–3.4%. Replacing it with a Graph CAM still led to a 1.2%–2.5% drop, indicating its critical role in highlighting essential nodes for effective adversarial manipulation. Similarly, the exclusion of malicious paths led to a reduction in our approach's ability to strategically perturb the graph. This outcome underscores the importance of these paths in identifying and exploiting vulnerabilities within the graph structure. Thus, both components are integral to maintaining the potency and precision of our adversarial strategies.

Impact of different types of benign nops: When evaluating our approach against the target model on the ADFA-LD dataset, we explored the impact of various benign syscalls by analyzing their frequency of occurrence during the rewiring operations, as illustrated in Fig. 5. Notably, certain benign syscalls, such as "read" and "getuid", appeared more frequently, indicating their effectiveness in facilitating successful evasion. Conversely, others like "mknodat" and "readlinkat" were less common. Fig. 5 highlights the importance of selecting specific types of benign syscalls in our approach. Consequently, we ultimately restricted our strategy to the 5 most commonly used operations to refine our approach.

Impact of the hyper-parameters in our approach: We investigate the impact of the hyper-parameters for our approach. Initially, for the number of the important node C, we adhere to the implementation settings, varying C to 6, 8,10, 12, 14, and 16. As illustrated in Fig. 6, it is observed that as C increases from 6 to 10 the ASRs (Attack Success Rates) of our approach rise

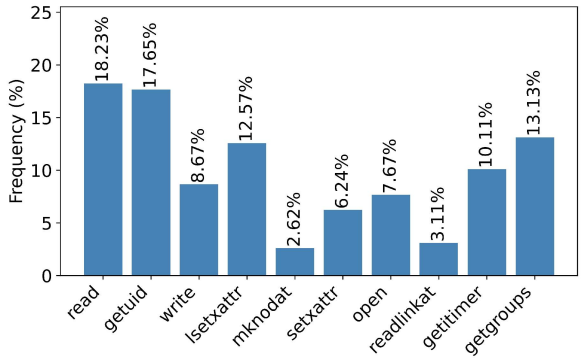

Fig. 5. Occurrence frequency of different benign syscalls that lead to successful evasions.

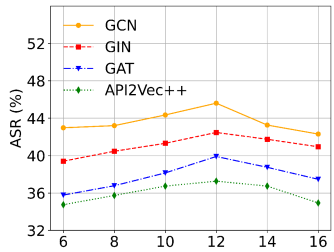

Fig. 6. Impact of the important node number.

TABLE III
PERTURBATION RATE OF DIFFERENT ATTACK METHODS.

Datasets	Method	Perturbation rate
AndroCT	Random	35.56%
	ReWatt	30.24%
	PageRank	31.42%
	RL	32.18%
	CAMA	29.23%
	Our approach	28.61%
ADFA-LD	Random	31.28%
	ReWatt	27.65%
	PageRank	25.42%
	RL	27.32%
	CAMA	25.21%
	Our approach	24.78%

sharply up to over 37% against all four target systems. Once C is more than 12, it will damage the performance of approach.

Similarly, for the max trace length N of API/system call sequence, we vary N to values of 20, 40, 60, 80, 100, 120 to demonstrate its impact as shown in Fig. 7. When N equals 40, the overall attack performance of our approach against all four target detection systems is quite effective, with ASRs exceeding 37%. With an increase in N, the ASRs of our approach remain stable. These observations suggest that our approach has high robustness, which can achieve high attack performance with different hyper-parameters.

3) RQ3: Modification Efficiency Comparisons.: Our approach employs heuristic random walks and strategically generated subgraphs, enhanced by reinforcement learning techniques

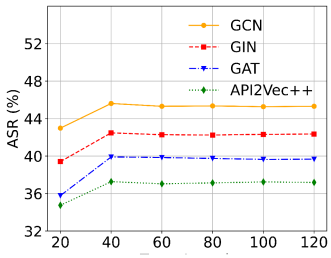

Fig. 7. Impact of the trace length.

TABLE IV
ATTACK SUCCESS RATE FOR THE DIFFERENT DETECTION METHODS

Datasets	Method	Success rate
AndroCT	LSTM-based Model	33.15%
	Transformer-based Model	27.62%
	CNN-based Model	32.33%
ADFA-LD	LSTM-based Model	39.41%
	Transformer-based Model	32.31%
	CNN-based Model	30.78%

to optimize graph modifications. This method has demonstrated lower perturbation rates on datasets like AndroCT, where we achieved a perturbation rate of only 28.1%. Unlike methods such as random ReWatt, our strategy focuses on global-graph-level perturbations, selectively targeting areas that significantly influence the graph classification tasks.

On the other hand, the reinforcement learning component specifically tunes the selection of edges and optimal benign nodes (API/syscall) for insertion, enhancing the perturbation's effectiveness while minimizing its visibility. This targeted approach ensures that changes are not only effective but also strategically placed to avoid detection by graph-based malware detectors. On the AndroCT dataset, for example, our approach not only maintains the functional integrity of the application but also ensures that the adversarial modifications are subtle yet significant enough to evade the detection system. The careful selection of benign system calls, which appear more frequently in successful evasions, underscores the importance of choosing specific types of operations that blend seamlessly into the normal operational profile of the application.

4) RQ4: Generalizability: We also investigated our our adversarial attack approach for rewiring operations to challenge API/system call sequence-based malware detection models. The premise of our study assumes that modifications at the graph level obscure the dependencies between API/system calls, thereby maintaining their adversarial nature.

To evaluate this hypothesis, we conducted experiments against three prominent sequence-based malware detection models: LSTM-based [55], Transformer-based [56], and CNN-based models [57]. These models were chosen for their strong performance in capturing a range of sequence dependencies, which provides a comprehensive basis for assessing the effectiveness of our attack across different sequence-based approaches.

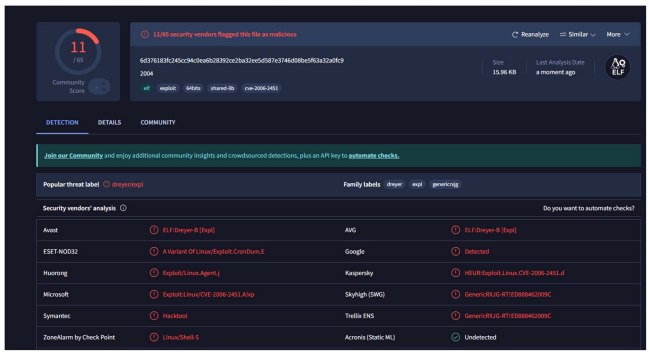

Fig. 8. VirusTotal detection results for executable generated from 2004.c (original version).

The results, as illustrated in the Table IV, show that our adversarial sequences achieve significant success rates against all tested models, suggesting that the rewiring strategy not only disrupts graph-based detection but also effectively translates its impact into the sequential domain.

Specifically, each of the three sequence-based models demonstrates varying degrees of susceptibility to the adversarial modifications, achieving notable bypass rates across datasets. For instance, on the AndroCT dataset, our approach yields a success rate of 27.62% on the Transformer-based model, while the LSTM-based and CNN-based models achieve success rates of 33.15% and 32.33%, respectively.

These findings underscore the versatility of our approach, as the adversarial modifications—initially crafted to disrupt graph-based detection models—retain adversarial characteristics that are equally effective in challenging sequence-based malware detection systems. Specifically, the rewiring operations, once translated into modified API/system call sequences, also obscure the sequential dependencies that sequence-based detectors rely on for classification. This also reveals a fundamental vulnerability in sequence-based models when faced with adversarial transformations originating at the graph level.

Impact of Surrogate Model Quality on Attack Transferability: To assess the feasibility of using surrogate models in a realistic closed-box setting, we systematically investigated how the training quality and architecture of the surrogate model affect attack performance on the ADFA-LD dataset.

1) Effect of Training Accuracy: We trained multiple surrogate models with varying levels of accuracy, ranging from 75.2% to 89.6%, by controlling the number of training epochs, initialization seeds. Each surrogate was then used to generate adversarial perturbations, which were applied to the fixed target model (GCN) to evaluate the attack success rate. As shown in Table V, higher surrogate accuracy consistently led to better transferability, with SR increasing 31.2% to 41.3%. This indicates that better learning of the data distribution and label mapping allows the surrogate model to produce more effective and transferable perturbations.

2) Effect of Architecture and Hyperparameters: We further analyzed the impact of key architectural parameters, including the number of layers, hidden dimensions. Surrogate models with 2, 3, and 4 GNN layers were evaluated, along with hidden sizes

TABLE V
EFFECTS OF SURROGATE MODEL QUALITY FACTORS ON ATTACK SUCCESS RATE

Factor	Configuration	Surrogate Acc. (%)	ASR on Target (%)
Training Accuracy	Epochs = 50	75.2	31.2
	Epochs = 100	80.5	33.1
	Epochs = 150	85.7	37.6
	Epochs = 200	89.6	41.3
Architecture	2-layer GCN, 64 dim	85.1	35.2
	3-layer GCN, 64 dim	85.5	37.9
	4-layer GCN, 64 dim	85.8	38.1
	3-layer GCN, 128 dim	86.2	39.3
Query Budget	500 queries	77.3	32.6
	1,000 queries	84.3	36.7
	2,000 queries	88.2	40.1

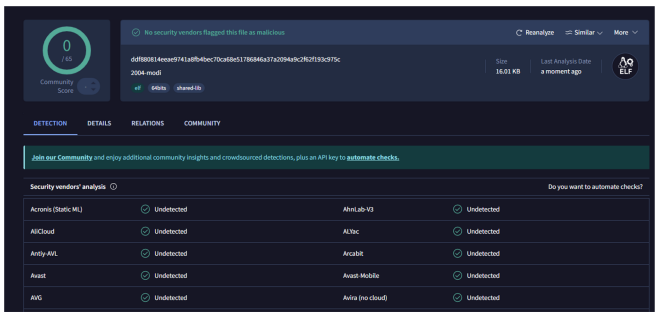

Fig. 9. VirusTotal detection results for executable generated from 2004.c (modified version)

of 32, 64, 128 and 256. While different configurations resulted in slight variations in attack success rates, all settings achieved reasonably good transferability. This suggests that, under our setup, surrogate models with the configurations are generally sufficient to capture the structural patterns necessary for effective closed-box transfer attacks.

3) Query Budget: All surrogate models were trained under a limited query budget. Specifically, when reducing the number of queries from 2,000 to 500, the attack success rate decreased from 40.1% to 32.6%, reflecting a drop of approximately 7.5%. This suggests that our approach remains effective even under constrained query conditions, demonstrating robustness against query limitations.

Overall, these experiments demonstrate that the transferability of attacks is influenced by the surrogate model's training accuracy, architectural parameter configuration, and the available query budget. We believe this finding deepens the understanding of transfer attack mechanisms on graph-structured data and provides valuable guidance for designing more powerful and generalizable attack strategies under realistic closed-box constraints.

The results show that, despite the insertion of interfering benign system calls, the core attack functionality of the sample remains intact. This demonstrates that our perturbation method successfully modifies the graph structure while preserving semantic consistency.

5) RQ5: Utility Performance: We also conduct a case study to demonstrate how the attacker modifies the graph structure through rewiring operations, as shown in Fig. 8 and Fig. 9.

In our study, we begin by compiling and tracing the execution of a malware from Exploit-DB, using the strace tool, resulting in an initial system call sequence: {`openat, mmap, getpid, clone, getpid, mmap, clock_nanosleep, exit_group`}. From this sequence, a system call graph is constructed to visualize the relationships of calls.

After constructing the graph, we identify key subgraphs and malicious paths. One such identified key subgraph includes {`clone, getpid,mmap, clock_nanosleep`}, where can be identified frequent interactions and critical parts within the execution.

Then our approach use reinforcement learning to determine optimal insertion points. The approach also insert `setxattr` adjacent to each mmap call and `getuid` next to each clone call. This method inserts these common and low-risk calls into the execution flow, thereby complicating the system call graph, challenging the pattern recognition capabilities of malware detection systems

To implement this modification, the program source code is first compiled into LLVM IR bitcode. Then, the LLVM IR bitcode is modified by automatically inserting custom external functions before each statement that may generate expected system call. Finally, the modified LLVM IR bitcode is recompiled into an executable file and generates the expected system call sequence. The result underscores the feasibility of our approach in manipulating the call graph of a malware.

Overall, prior adversarial attacks either only generate non-executable adversarial "features". However, our approach exhibits the high performance while preserves the original malware functionality.

In addition to evaluating adversarial examples against graph-based malware classifiers that rely on structural representations extracted from dynamic analysis (e.g., API or system call graphs processed by Graph Neural Networks, GNNs), we further assessed their detection evasion capabilities through VirusTotal. We produced and submitted 106 adversarial samples from exploit-db, each of which is an executable reconstructed after benign syscall insertion. The PoCs cover 5 types of vulnerabilities, with the following distribution: 49 Denial-of-Service PoCs, 33 Privilege Escalation PoCs, 14 Information Disclosure PoCs, 6 Memory Disclosure PoCs, and 4 Security Bypass PoCs. Before modification, all original samples were successfully detected as malicious by multiple mainstream antivirus engines on

VirusTotal, with an average of 10 to 25 engines flagging each sample. 11 adversarial samples were completely undetected by any antivirus engine on VirusTotal. 39 adversarial samples triggered at least 30% fewer detection engines compared to their original counterparts. These findings indicate that the proposed modifications can undermine the detection capability of multiple engines, thereby demonstrating the practical relevance of our approach even beyond the GNN-based evaluation setting. Representative VirusTotal detection results before and after modification are provided below.

Functional Consistency Verification Procedure: To ensure that adversarial samples preserved their original attack functionality after inserting or replacing system calls, we checked the behavior of each sample both before and after modification. Real-world exploits often involve multiple attack objectives, such as privilege escalation, file manipulation, reverse shell creation, and system configuration changes. We executed the samples within a sandbox environment. We consider an adversarial sample to preserve its functionality if it still triggers the same attack effect as the original sample. we assessed whether key malicious behaviors—such as privilege escalation, file manipulation, or network communications—were preserved, among other potential attack effects:

1) *Privilege escalation:* We monitor the UID of the executing process; a UID of 0 indicates successful escalation.

2) *File system operations:* We determine whether target files (e.g., `/tmp/backdoor.txt`) are created or modified, and verify content integrity using hash comparison.

3) *Network behavior:* We check whether a reverse shell is initiated or whether any predefined ports become active during execution.

4) *System state modification:* For example, in the case of new user creation, we compare the `/etc/passwd` file before and after execution to check for changes.

Across all evaluated samples, we observed no functional deviation between the original and modified versions under the above criteria. This suggests that our perturbation strategy effectively alters graph structure while empirically preserving the core attack semantics.

In addition, we evaluate the computational efficiency of our proposed attack, We separate the computational cost into a one-time training stage and a per-sample generation stage. In the training stage, the surrogate model and the reinforcement learning policy network are trained using a fixed query budget of 2,000 queries to the target model, which is a one-time cost and shared across all attack samples. On our experimental platform, this stage takes approximately 15–20 minutes in total. During the generation stage, no further training is required; generating one adversarial sample only involves forward inference and limited rewiring operations, taking about 8–12 seconds on average (excluding dynamic tracing). This demonstrates that the proposed attack is computationally practical in multi-sample settings.

6) RQ6: Effectiveness Under Reduction Strategies: In practice, malware detection systems—particularly those based on dynamic analysis—often do not operate directly on raw API or system call graphs. Instead, various abstraction, filtering, and behavior reduction techniques are typically employed during graph construction to highlight semantically significant malicious behaviors while filtering out some benign or noisy behaviors. These preprocessing strategies may partially nullify the benign call patterns introduced by our attack, potentially reducing its effectiveness before the graph is passed to the GNN-based detector.

To evaluate the robustness of our approach under such realistic conditions, we assesses its performance when applied to reduction strategies. Inspired by prior work [58], [59], [60], we implement the following five reduction strategies:

1) Semantic Type Mapping: We map raw API calls (including their arguments) to high-level semantic categories. For example, `ReadFile` and `WriteFile` are both mapped to the *File Operation* type. This abstraction reduces node diversity.

2) Redundant Path Merging: We merge repeated or structurally equivalent API call sequences. This removes redundant substructures—such as identical call chains within loops—and simplifies the overall graph topology.

3) Low-Entropy Call Filtering: We filter out statistically low-information API calls. This reduces noise and allows the model to focus on semantically meaningful behavioral patterns.

4) Node Version Control: We assign versioned node types to API calls based on their temporal order (e.g., `Retrofit_v1`, `Retrofit_v2`). While our attack is performed on the original graph without versioning, this strategy reconstructs the graph with time-aware node types. Inserted calls are thus treated as structurally distinct nodes, potentially reducing their effect on the model.

5) Core Subgraph Preservation: We retain only the most informative subgraphs based on structural importance scores (e.g., PageRank or other centrality metrics). This ensures that high-confidence behavioral regions are preserved while peripheral or low-relevance parts of the graph are discarded.

We examined the graph construction procedures of the target detection models to determine whether any built-in reduction strategies were employed. The GCN-based model corresponds to a *Low-Entropy Call Filtering* strategy in our taxonomy. For GAT-based and GIN-based models, they do not describe any form of graph reduction. We therefore assume these models operate directly on the unfiltered raw graphs. In contrast, API2vec++ incorporates multiple graph preprocessing steps. Specifically, it filters out low-value or redundant edges, which corresponds to the effect of our *Redundant Path Merging* strategy, and constructs time-ordered (happen-before) edges with logical timestamps, which achieves a similar purpose to our *Node Version Control* strategy by organizing multiple change sequences based on logical time.

Therefore, for each target model, if a strategy is already present, we retain the model's built-in preprocessing. For all remaining strategies that were not originally employed by the model, we apply them individually to evaluate their defensive effect. In every case, adversarial perturbations are applied to the original graph before any reduction, and the model is trained and tested on the reduced graph. We conduct these evaluations on the AndroCT dataset, and the results are summarized in Table VI.

TABLE VI
DECREASE IN ATTACK SUCCESS RATE UNDER DIFFERENT GRAPH REDUCTION STRATEGIES

strategies	GCN	GIN	GAT	API2vec++
Semantic Type Mapping	6.38%	6.04%	6.21%	5.73%
Redundant Path Merging	5.42%	5.03%	4.94%	—
Low-Entropy API Filtering	—	2.74%	3.13%	3.22%
Node Version Control	8.71%	8.18%	8.42%	—
Core-Subgraph Extraction	7.11%	6.79%	6.93%	7.22%

Table VI reports the drop in attack success rate when different graph reduction strategies are applied to the perturbed call graphs. While all strategies lead to some performance degradation, the reductions are consistently modest—typically under 9%—which demonstrates that our approach remains effective even under such defensive transformations.

Our approach prioritizes high-importance subgraphs that significantly influence the model's decision boundary. Through node significance analysis and heuristic random walks, we identify behaviorally critical regions. Reinforcement-learning-based rewiring then injects benign yet semantically compatible operations into these influential regions, ensuring that adversarial modifications survive graph simplification. As a result, even when the graph is simplified—e.g., via node filtering, path merging, or semantic abstraction—many of the adversarial modifications remain intact within the retained substructure.

Moreover, our approach deliberately avoids reliance on low-entropy or redundant nodes, which are typically removed during reduction. For instance, under Low-Entropy API Filtering, our attack degrades by only 2–4%, confirming that our modifications are not concentrated in low-value regions. Similarly, the relatively small impact of Semantic Type Mapping and Redundant Path Merging suggests that our rewiring is not sensitive to fine-grained call diversity or structural repetition.

Node version control also reduces attack effectiveness to some extent, yet our approach remains effective because the perturbations modify decision-critical semantic–structural patterns that persist under temporal modeling. The inserted calls are temporally consistent in real execution and reshape the contexts of time-aware path embeddings, diluting suspicious co-occurrences and altering the resulting path and graph representations.

In summary, the consistent performance of our attack across all reduction strategies highlights its robustness and practical viability. Even under realistic preprocessing steps used by malware detectors to reduce noise and emphasize core behavior, the attack success rate only drops by a relatively small margin across all target models.

V. RELATED WORK

A. Graph-Learning-Based Malware Detection

In recent advancements in malware detection, various graph-based neural network models have been developed to enhance the identification and classification of malware across different systems. In 2020, John et al. introduced a method using Graph Convolutional Networks (GCN) for Android malware detection, which models system call sequences as graphs to capture structural dependencies [50]. Following this, Deng et al. in 2023 developed "Enimanal," a framework utilizing Graph Isomorphism Networks (GIN) for analyzing IoT malware by integrating detailed information from the Linux Programmer Manual into system call graphs [51]. In 2024, Feng et al. presented DawnGNN, a Windows malware detection framework that combines official API documentation with a Graph Attention Network (GAT) enhanced by a pre-trained BERT model to improve the semantic representation of API calls [13]. Additionally, the API2Vec++ method employs a heuristic random walk on Temporal Process and API Property Graphs, using BERT for embedding paths to detect complex multi-process malware behaviors effectively [12]. These models collectively represent significant strides in leveraging graph neural networks for cybersecurity.

B. Adversarial Attack on Graph Data

Adversarial attacks on graph-structured data aims to test and improve the robustness of graph neural networks. In 2018, Dai et al. presented a simple yet sometimes effective method that involves randomly selecting nodes and edges to perturb, which requires minimal information for execution [19]. Additionally, the same study developed a more sophisticated approach using reinforcement learning, which employs a hierarchical Q-function to manage the quadratic action space of adding or deleting edges [19]. This method simplifies the decision-making process in edge modification, enhancing the training efficiency of the reinforcement learning model. In 2021, Ma introduced the ReWatt technique, which conducts rewiring operations by removing an edge between a node and its direct neighbor and connecting it to a more distant one, thus effectively altering the graph structure but potentially disrupting the original functionality [53]. Furthermore, the PageRank method targets nodes with the highest centrality metrics for attack, exploiting their influential positions within the network [24]. These methods collectively highlight the diverse strategies employed to manipulate and challenge graph-based systems.

VI. DISCUSSION

Limitations: First, our approach may need continual updates to keep pace with rapidly evolving malware. Furthermore, the call graph construction in this paper is a relatively simple and commonly used method, while some detection system call graph construction methods [9], [12] are more complex and utilize some prior information, which may affect the performance of attacks. We will discuss the impact of different graph construction methods on attack performance in our future work. Additionally, pre-training a surrogate model via transfer learning to reduce the query budget is a promising direction we plan to explore.

Potential Defenses: Our attack also reveals several promising defense directions. These include adversarial training with attack-generated samples to learn more robust decision boundaries, graph consistency checks that verify whether inserted nodes exhibit realistic argument distributions and temporal ordering, semantic constraint enforcement to flag contextually

implausible syscall insertions, graph reduction techniques to abstract away superficial modifications, and ensemble detection combining static and dynamic analysis perspectives. We believe effective defense likely requires a combination of these strategies rather than any single mechanism in isolation.

VII. Conclusion

Through the deployment of a practical closed-box adversarial attack, our research underscores the potential risks associated with current malware detection frameworks that rely on API/system call graphs. By integrating techniques such as subgraph extraction, heuristic random walks, and reinforcement learning-driven rewiring, our approach not only obfuscates malicious intents to evade detection effectively but also ensures that the adversarial malware retains its original functionality. The comprehensive evaluation across multiple state-of-the-art graph-based malware detection systems reveals a high success rate for these attacks, confirming the critical need for advancements in security measures to counter such vulnerabilities.

References

[1] T. Bilot, N. El Madhoun, K. Al Agha, and A. Zouaoui, "A survey on malware detection with graph representation learning," *ACM Comput. Surv.*, vol. 56, no. 11, pp. 1–36, 2024.

[2] M. Saqib, S. Mahdavifar, B. C. Fung, and P. Charland, "A comprehensive analysis of explainable AI for malware hunting," *ACM Comput. Surv.*, vol. 56, 2024, Art. no. 314.

[3] S. Rohini, G. Ramesh, and A. R. Nair, "MAGIC: Malware behaviour analysis and impact quantification through signature co-occurrence and regression," *Comput. Secur.*, vol. 139, 2024, Art. no. 103735.

[4] X. Ling et al., "A wolf in sheep's clothing: Practical black-box adversarial attacks for evading learning-based windows malware detection in the wild," in *Proc. 33rd USENIX Secur. Symp.*, 2024, pp. 7393–7410.

[5] D. Zapzalka, S. Salem, and D. Mohaisen, "Semantics-preserving node injection attacks against GNN-based ACFG malware classifiers," *IEEE Trans. Dependable Secure Comput.*, vol. 22, no. 1, pp. 549–560, Jan./Feb. 2025.

[6] O. Kargarnovin, A. M. Sadeghzadeh, and R. Jalili, "MAL2GCN: A robust malware detection approach using deep graph convolutional networks with non-negative weights," *J. Comput. Virol. Hacking Techn.*, vol. 20, no. 1, pp. 95–111, 2024.

[7] S. Gulmez, A. G. Kakisim, and I. Sogukpinar, "Xran: Explainable deep learning-based ransomware detection using dynamic analysis," *Comput. Secur.*, vol. 139, 2024, Art. no. 103703.

[8] D. Trizna, L. Demetrio, B. Biggio, and F. Roli, "Nebula: Self-attention for dynamic malware analysis," *IEEE Trans. Inf. Forensics Secur.*, vol. 19, pp. 6155–6167, 2024.

[9] L. Cui, J. Cui, Y. Ji, Z. Hao, L. Li, and Z. Ding, "API2VEC: Learning representations of API sequences for malware detection," in *Proc. 32nd ACM SIGSOFT Int. Symp. Softw. Testing Anal.*, 2023, pp. 261–273.

[10] K. Tan, D. Zhan, L. Ye, H. Zhang, and B. Fang, "A practical adversarial attack against sequence-based deep learning malware classifiers," *IEEE Trans. Comput.*, vol. 73, no. 3, pp. 708–721, Mar. 2023.

[11] D. Zhan, K. Tan, L. Ye, X. Yu, H. Zhang, and Z. He, "An adversarial robust behavior sequence anomaly detection approach based on critical behavior unit learning," *IEEE Trans. Comput.*, vol. 72, no. 11, pp. 3286–3299, Nov. 2023.

[12] L. Cui et al., "API2Vec : Boosting API sequence representation for malware detection and classification," *IEEE Trans. Softw. Eng.*, vol. 50, no. 8, pp. 2142–2162, Aug. 2024.

[13] P. Feng et al., "DawnGNN: Documentation augmented windows malware detection using graph neural network," *Comput. Secur.*, vol. 140, 2024, Art. no. 103788.

[14] C. Li et al., "DMalNet: Dynamic malware analysis based on API feature engineering and graph learning," *Comput. Secur.*, vol. 122, 2022, Art. no. 102872.

[15] T. Chen, H. Zeng, M. Lv, and T. Zhu, "CTIMD: Cyber threat intelligence enhanced malware detection using API call sequences with parameters," *Comput. Secur.*, vol. 136, 2024, Art. no. 103518.

[16] C. Liu, B. Li, X. Liu, C. Li, and J. Bao, "Evolving malware detection through instant dynamic graph inverse reinforcement learning," *Knowledge-Based Syst.*, vol. 299, 2024, Art. no. 111991.

[17] Z. Liu, R. Wang, N. Japkowicz, H. M. Gomes, B. Peng, and W. Zhang, "SeGDroid: An android malware detection method based on sensitive function call graph learning," *Expert Syst. Appl.*, vol. 235, 2024, Art. no. 121125.

[18] H. Chang et al., "A restricted black-box adversarial framework towards attacking graph embedding models," in *Proc. AAAI Conf. Artif. Intell.*, vol. 34, no. 04, 2020, pp. 3389–3396.

[19] H. Dai et al., "Adversarial attack on graph structured data," in *Proc. Int. Conf. Mach. Learn.*. 2018, pp. 1115–1124.

[20] Y. Xu, M. Lanier, A. Sarkar, and Y. Vorobeychik, "Attacks on node attributes in graph neural networks," 2024, *arXiv:2402.12426*.

[21] X. Wang et al., "Revisiting adversarial attacks on graph neural networks for graph classification," *IEEE Trans. Knowl. Data Eng.*, vol. 36, no. 5, pp. 2166–2178, May 2024.

[22] L. Wen, J. Liang, K. Yao, and Z. Wang, "Black-box adversarial attack on graph neural networks with node voting mechanism," *IEEE Trans. Knowl. Data Eng.*, vol. 36, no. 10, pp. 5025–5038, Oct. 2024.

[23] Y. Chen, Z. Ye, Z. Wang, and H. Zhao, "Imperceptible graph injection attack on graph neural networks," *Complex Intell. Syst.*, vol. 10, no. 1, pp. 869–883, 2024.

[24] J. Ma, S. Ding, and Q. Mei, "Towards more practical adversarial attacks on graph neural networks," in *Proc. Adv. Neural Inf. Process. Syst.*, 2020, vol. 33, pp. 4756–4766.

[25] A. Venturi, D. Stabili, and M. Marchetti, "Problem space structural adversarial attacks for network intrusion detection systems based on graph neural networks," 2024, *arXiv:2403.11830*.

[26] G. Zhu, M. Chen, C. Yuan, and Y. Huang, "Simple and efficient partial graph adversarial attack: A new perspective," *IEEE Trans. Knowl. Data Eng.*, vol. 36, no. 8, pp. 4245–4259, Aug. 2024.

[27] X. Zou et al., "TDGIA: Effective injection attacks on graph neural networks," in *Proc. 27th ACM SIGKDD Conf. Knowl. Discov. Data Mining*, 2021, pp. 2461–2471.

[28] K. Xu et al., "Topology attack and defense for graph neural networks: An optimization perspective," in *Proc. 28th Int. Joint Conf. Artif. Intell.*, 2019, pp. 3961–3967.

[29] J. Liao, W. Fu, C. Wang, Z. Wei, and J. Xu, "Value at adversarial risk: A graph defense strategy against cost-aware attacks," in *Proc. AAAI Conf. Artif. Intell.*, vol. 38, no. 12, 2024, pp. 13763–13 771.

[30] K. Zhao et al., "Structural attack against graph based android malware detection," in *Proc. ACM SIGSAC Conf. Comput. Commun. Secur.*, 2021, pp. 3218–3235.

[31] A. Abusnaina, A. Khormali, H. Alasmary, J. Park, A. Anwar, and A. Mohaisen, "Adversarial learning attacks on graph-based iot malware detection systems," in *Proc. IEEE 39th Int. Conf. Distrib. Comput. Syst.*, 2019, pp. 1296–1305.

[32] S. Pandey, N. Kumar, A. Handa, and S. K. Shukla, "Evading malware classifiers using RL agent with action-mask," *Int. J. Inf. Secur.*, vol. 22, no. 6, pp. 1743–1763, 2023.

[33] M. Rigaki and S. Garcia, "The power of MEME: Adversarial malware creation with model-based reinforcement learning," in *Proc. Eur. Symp. Res. Comput. Secur.*, 2023, pp. 44–64.

[34] L. Chen, Y. Ye, and T. Bourlai, "Adversarial machine learning in malware detection: Arms race between evasion attack and defense," in *Proc. Eur. Intell. Secur. Informat. Conf.*, 2017, pp. 99–106.

[35] K. Lucas, M. Sharif, L. Bauer, M. K. Reiter, and S. Shintre, "Malware makeover: Breaking ML-based static analysis by modifying executable bytes," in *Proc. ACM Asia Conf. Comput. Commun. Secur.*, 2021, pp. 744–758.

[36] Y. Sun, S. Wang, X. Tang, T.-Y. Hsieh, and V. Honavar, "Non-target-specific node injection attacks on graph neural networks: A hierarchical reinforcement learning approach," in *Proc. Web Conf.*, vol. 3, 2020, pp. 673–683.

[37] D. Chen et al., "Single-node injection label specificity attack on graph neural networks via reinforcement learning," *IEEE Trans. Computat. Social Syst.*, vol. 11, no. 5, pp. 6135–6150, Oct. 2024.

[38] S. Zhang, H. Tong, J. Xu, and R. Maciejewski, "Graph convolutional networks: A comprehensive review," *Comput. Social Netw.*, vol. 6, no. 1, pp. 1–23, 2019.

[39] A. Al-Dujaili, A. Huang, E. Hemberg, and U.-M. O' Reilly, "Adversarial deep learning for robust detection of binary encoded malware," in *Proc. IEEE Secur. Privacy Workshops*, 2018, pp. 76–82.
[40] B. Kolosnjaji et al., "Adversarial malware binaries: Evading deep learning for malware detection in executables," in *Proc. 26th Eur. Signal Process. Conf.*, 2018, pp. 533–537.
[41] B. Zhou, A. Khosla, A. Lapedriza, A. Oliva, and A. Torralba, "Learning deep features for discriminative localization," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, 2016, pp. 2921–2929.
[42] P. E. Pope, S. Kolouri, M. Rostami, C. E. Martin, and H. Hoffmann, "Explainability methods for graph convolutional neural networks," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, 2019, pp. 10772–10781.
[43] H. Wang et al., "Score-cam: Score-weighted visual explanations for convolutional neural networks," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. workshops*, 2020, pp. 24–25.
[44] F. Xia, J. Liu, H. Nie, Y. Fu, L. Wan, and X. Kong, "Random walks: A review of algorithms and applications," *IEEE Trans. Emerg. Topics Comput. Intell.*, vol. 4, no. 2, pp. 95–107, Apr. 2020.
[45] D. Ernst and L. Arthur, "Introduction to reinforcement learning," *Feuerriegel, S., Hartmann, J., Janiesch, C., and Zschech, P*, 2024, pp. 111–126.
[46] R. S. Sutton, D. McAllester, S. Singh, and Y. Mansour, "Policy gradient methods for reinforcement learning with function approximation," in *Proc. Adv. Neural Inf. Process. Syst.*, 1999, vol. 12, pp. 1057–1063.
[47] C. Lattner and V. Adve, "LLVM: A compilation framework for lifelong program analysis & transformation," in *Proc. Int. Symp. Code Gener. Optim.*, 2004, 2004, pp. 75–86.
[48] G. Creech and J. Hu, "Generation of a new IDs test dataset: Time to retire the KDD collection," in *Proc. IEEE Wireless Commun. Netw. Conf.*, 2013, pp. 4487–4492.
[49] W. Li, X. Fu, and H. Cai, "AndroCT: Ten years of app call traces in android," in *Proc. IEEE/ACM 18th Int. Conf. Mining Softw. Repositories*, 2021, pp. 570–574.
[50] T. S. John, T. Thomas, and S. Emmanuel, "Graph convolutional networks for android malware detection with system call graphs," in *Proc. 3rd ISEA Conf. Secur. Privacy*, 2020, pp. 162–170.
[51] L. Deng, H. Wen, M. Xin, H. Li, Z. Pan, and L. Sun, "Enimanal: Augmented cross-architecture IoT malware analysis using graph neural networks," *Comput. Secur.*, vol. 132, 2023, Art. no. 103323.
[52] D. Zügner, A. Akbarnejad, and S. Günnemann, "Adversarial attacks on neural networks for graph data," in *Proc. 24th ACM SIGKDD Int. Conf. Knowl. Discov. Data Mining*, 2018, pp. 2847–2856.
[53] Y. Ma, S. Wang, T. Derr, L. Wu, and J. Tang, "Graph adversarial attack via rewiring," in *Proc. 27th ACM SIGKDD Conf. Knowl. Discov. Data Mining*, 2021, pp. 1161–1169.
[54] X. Wang et al., "Revisiting adversarial attacks on graph neural networks for graph classification," *IEEE Trans. Knowl. Data Eng.*, vol. 36, no. 05, pp. 2166–2178, May 2024.
[55] M. Du, F. Li, G. Zheng, and V. Srikumar, "DeepLog: Anomaly detection and diagnosis from system logs through deep learning," in *Proc. ACM SIGSAC Conf. Comput. Commun. Secur.*, 2017, pp. 1285–1298.
[56] S. Nedelkoski, J. Bogatinovski, A. Acker, J. Cardoso, and O. Kao, "Self-attentive classification-based anomaly detection in unstructured logs," in *IEEE Int. Conf. Data Mining*, 2020, pp. 1196–1201.
[57] S. Lu, X. Wei, Y. Li, and L. Wang, "Detecting anomaly in Big Data system logs using convolutional neural network," in *Proc. IEEE 16th Int. Conf. Dependable, Auton. Secure Comput., 16th Int. Conf. Pervasive Intell. Comput., 4th Int. Conf. Big Data Intell. Comput. Cyber Sci. Technol.*, 2018, pp. 151–158.
[58] S. M. Milajerdi, B. Eshete, R. Gjomemo, and V. Venkatakrishnan, "POIROT: Aligning attack behavior with kernel audit records for cyber threat hunting," in *Proc. ACM SIGSAC Conf. Comput. Commun. Secur.*, 2019, pp. 1795–1812.
[59] E. Altinisik, F. Deniz, and H. T. Sencar, "PROVG-Searcher: A graph representation learning approach for efficient provenance graph search," in *Proc. ACM SIGSAC Conf. Comput. Commun. Secur.*, 2023, pp. 2247–2261.
[60] R. Wei, L. Cai, L. Zhao, A. Yu, and D. Meng, "DeepHunter: A graph neural network based approach for robust cyber threat hunting," in *Proc. Int. Conf. Secur. Privacy Commun. Syst.*, 2021, pp. 3–24.

Kai Tan received the PhD degree in cyberspace science from the Harbin Institute of Technology, China. He is currently an associate researcher with the School of Cyberspace Science, Harbin Institute of Technology. His research interests include cloud security and malware detection.

Dongyang Zhan (Member, IEEE) received the BS degree in computer science from the Harbin Institute of Technology (HIT), in 2014, and the PhD degree from the School of Computer Science and Technology, HIT, in 2019. He is currently an assistant professor with the School of Cyberspace Science, Harbin Institute of Technology. His research interests include cloud computing and security.

Haining Yu received the BS and MS degrees in computer science and the PhD degree in information security from the Harbin Institute of Technology, Harbin, China, in 2006, 2008, and 2013, respectively. He is currently a professor with the School of Cyberspace Science, Harbin Institute of Technology. He has authored or coauthored in *IEEE Transactions on Dependable and Secure Computing*, *IEEE Transactions on Information Forensics and Security*, and *IEEE Transactions on Services Computing*. His research interests include data security, privacy computing, and AI security.

Hongli Zhang (Member, IEEE) received the BS degree in computer science from Sichuan University, Chengdu, China, in 1994, and the PhD degree in computer science from the Harbin Institute of Technology (HIT), Harbin, China, in 1999. She is currently a professor with the School of Cyberspace Science, HIT. Her research interests include network and information security, network measurement and modeling, and parallel processing.

Bei Zhao received the BS and MS degrees in computer science from Northwestern Polytechnical University, Xi'an, China, in 1996 and 1999, respectively. She is currently a professor-level senior engineer with Cyber security Products Department, ChinaMobile Group Design Institute. She has authored or coauthored papers in journals, such as IEEE. Her research interests include network security, data security, and artificial intelligence security.

Xiaojun Jia received the PhD degree from the State Key Laboratory of Information Security, Institute of Information Engineering, Chinese Academy of Sciences, and also from the School of Cyber Security, University of Chinese Academy of Sciences, Beijing. He is currently a research fellow with Cyber Security Research Centre @ NTU, Nanyang Technological University, Singapore. His research interests include computer vision, deep learning, and adversarial machine learning.